TÉCNICO
UNIVERSIDADE
DE LISBOA

Digital Systems Design

MSc in Electrical and Computer Engineering

Lab. Proj. 1 – Simple Calculator

Authors:

Rodrigo Caldeira (106963) rodrigo.caldeira@tecnico.ulisboa.pt
Guilherme Garcia (105917) guilherme.f.garcia@tecnico.ulisboa.pt
Diogo Diniz (106196) diogo.cruz.diniz@tecnico.ulisboa.pt

Group 7

2026/2027 – 1st Semester

Contents

1 Introduction **2**

2 System architecture **2**

3 Interface **2**
3.1 Display and User Experience 2

4 Datapath **3**
4.1 Arithmetic Logic Unit (ALU) 3
4.2 Registers 4
4.3 Multiplexers 4
4.4 Design challenges 4

5 Control **4**
5.1 Inputs and outputs 4
5.2 State machine 4
5.3 Undefined states 5

6 Simulation **5**
6.1 Behavioral simulation 5

7 Synthesis results **6**

8 FPGA implementation **6**

9 Conclusions **6**

1 Introduction

This report presents the design, simulation, and hardware implementation of a digital calculator. The system was designed in VHDL and deployed on an AMD Artix-7 FPGA using the Basys 3 board.

The calculator supports four main operations: Addition, Multiplication, Shift Right Arithmetic (SRA) and Bitwise OR.

The system is composed of three main units: Interface, Datapath, and Control. The system displays real-time user feedback on a 7-segment display and LEDs, including arithmetic overflow detection.

Behavioral simulations verified all operational modes and overflow detection. The physical implementation achieved the following major results:

- Low resource usage: 111 Slice LUTs (0.53%), 168 registers (0.40%), and 1 DSP48E1 slice (1.11%).
- Successful timing closure at 100 MHz with a Worst Negative Slack (WNS) of 2.82 ns which implies a maximum achievable clock frequency of $f_{\max} = 139.28$ MHz.

2 System architecture

The system is composed by three units:

- **Interface unit (section 3):** The interface unit acts as the top-level wrapper, bridging physical FPGA I/O peripherals with the core processing circuit. It is responsible for debouncing button inputs, synchronizing signals to the main system clock, decoding commands, and driving the output displays.
- **Datapath unit (section 4):** The datapath unit implements the hardware needed to perform all the calculator operations. It consists of an ALU, two registers, and two multiplexers, along with the connections between these components. Figure 2 shows a diagram of the datapath unit along with all the data and control signals.
- **Control unit (section 5):** The control unit translates the user's high level input into a series of control signals for the datapath to perform the requested operation. It is composed of a state machine and a combinational logic block for operation decoding.

3 Interface

3.1 Display and User Experience

The interface mirrors the 16 slide switches on the LEDs and displays their live hexadecimal value on the 7-segment display, zero-extending the 15-bit input vector with an MSB '0'. This input preview persists until the equals or SRA button is pressed to display the operation result, while arithmetic overflow is signaled by illuminating all four display decimal points.

Figure 1 summarizes the physical interface layout, including button assignment, switch functions, operation encoding based on switch 15, and overflow behavior.



Figure 1: User interface layout showing button assignments, switch controls, and overflow handling.

4 Datapath

4.1 Arithmetic Logic Unit (ALU)

The most complex block present in the datapath is the ALU. As an arithmetic logic unit, it is responsible for performing the operations when given the operands. This block was built with generic usage in mind, allowing the same ALU to be used in many systems regardless of bus bit width. Since the current system uses 15 bit words, the unit is parameterized as such by default:

```
generic (N : integer := 15); -- alu operand width
```

The ALU contains four operators and a multiplexer (managed by `ALU_SEL`) that lets the control unit choose the operation that should be performed. It also evaluates overflow conditions and signals them to the interface (`ofl`), turning the decimal point segments of the display on and off accordingly.

Below is a succinct explanation of how to determine an overflow has occurred for each operation.

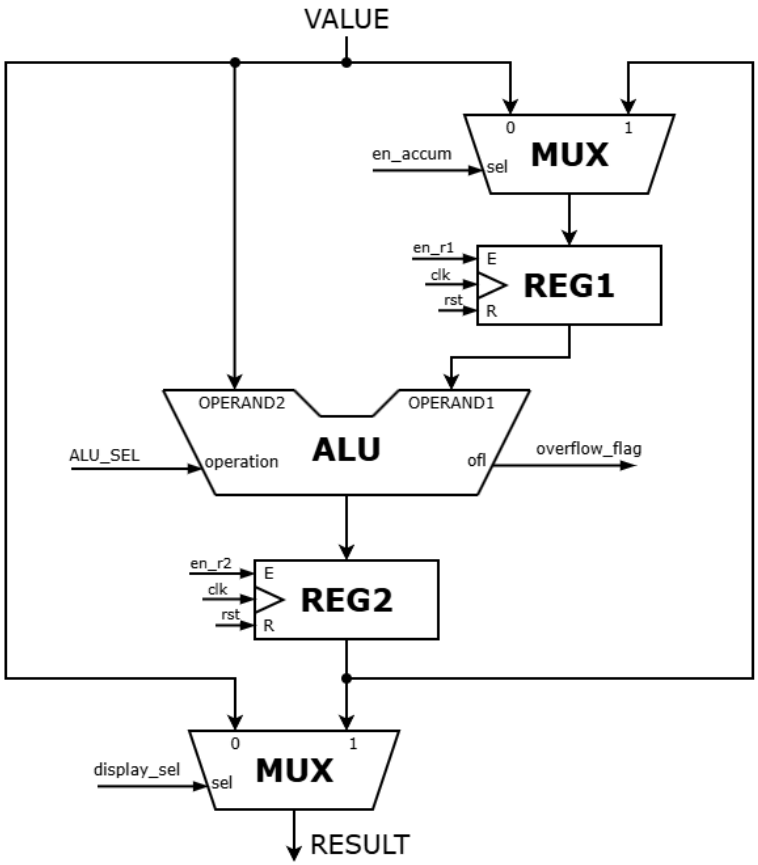


Figure 2: Diagram of the datapath unit showing all the hardware blocks, interconnections and the signals involved.

SUM: If the operands have the same sign but the result has a different sign, overflow has occurred. Otherwise, no overflow can ever happen. The sign comparisons are done with XOR operations.

MUL: If any of the sign extension bits is different from the result's MSB, there's overflow.

OR: Since this is a bit-wise operation, no overflow can happen.

SRA: Due to the nature of the shift operation, no overflow can happen. This operation simply rearranges bits in a fixed width word.

4.2 Registers

The datapath includes two registers, REG1 and REG2. These store the `OPERAND1` and the `ALU_RES`, respectively.

REG1 is essential to allow two operand operations since it can store the first operand while the second one is introduced by the user.

REG2 stores the result of the ALU which can be used later as an operand (enabling accumulator operation) or simply displayed by the interface, providing the user with the result of their calculation.

4.3 Multiplexers

The two multiplexers help connect the various blocks, performing a simple yet paramount function.

The multiplexer at the top of the diagram defines if the first operand of a given operation will be a value introduced by the user or rather the previous operation's result.

The bottom multiplexer allows the system to show on the display either the value introduced by the user (allowing them to check what was introduced as an operand) or the result of the operation upon pressing the equals button.

4.4 Design challenges

Performing the overflow detection while minimizing sequential statements was the most challenging part of this unit's development. Overflow conditions were determined and later converted into combinational logic instead of an `if...else` type statement, therefore keeping with good practices.

Another challenging issue for the datapath design was the fact that all operators except for the SRA expect two operands, while the latter only requires a single operand. This meant that some care had to be taken with the ALU's port mapping. `REG1` was assigned to `OPERAND1` and `VALUE` to `OPERAND2`. The SRA operation is always applied to `OPERAND1`, allowing it to be performed on either the VALUE or the accumulator through the use of the top MUX. The operand always comes from `REG1` because, prior to the user pressing the operation button, the system doesn't know what operation will be performed.

5 Control

5.1 Inputs and outputs

The control unit receives two main signals: the selected operation, encoded by the interface, and the debounced signal from the `equals` button. The encoded operation is a two bit selector paired with a `operation available` signal for repeated button pressing detection. The equals signal is a single bit, pulsed when the button is pressed.

To control the datapath, the control unit outputs five different ports: two enable signals, one per register, two MUX selectors, one for the display and one for `REG1` input selection, and one ALU operation selector composed of two bits.

5.2 State machine

For the implementation of the control section of the calculator, a Moore Finite State Machine was chosen. The simplicity and well defined separation of state transition from signal control logic result in ease of readability of the produced VHDL code.

The FSM defines six states:

- **Init:** The initial state of the FSM. Inserts the first operand.
- **Add:** Entered after pressing the `add` button. Used to insert the second operand.
- **Mul:** Entered after pressing the `multiply` button. Used to insert the second operand.
- **Or:** Entered after pressing the `or` button. Used to insert the second operand.

- **Shift Right Arithmetic (pt2):** Transitioned to after pressing the shift button. Since shift right takes no second operand, this state directly performs the shift operation and automatically transitions to the `Done` state.
- **Done:** The final state after any operation. Usually reached by pressing the `equals` button, but also driven by the execution of the shift operation. Displays the result of the operation, by now stored in `REG2`, and feeds it back to the `accumulator` for re-use in further operations.

Figure 3 shows a diagram of the defined states and their associated transitions.

5.3 Undefined states

For the definition of the states, three bits are used, which allows for eight states. Since only six of them are used, two values have undefined states. Although none of them are ever transitioned into, they were taken into account during design. If such a state were ever to be reached, it would be treated as another `Done` state and would not break the calculator.

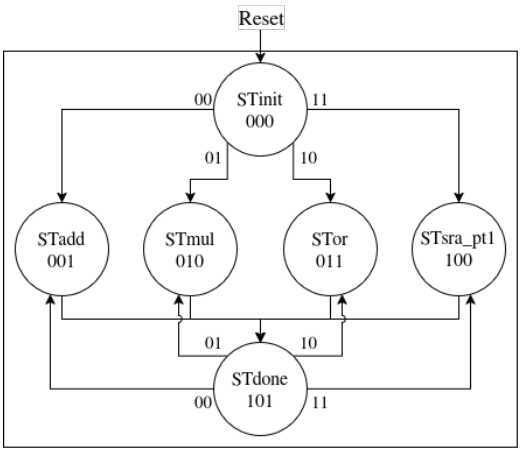


Figure 3: Diagram of the Finite State Machine implemented by `control`.

6 Simulation

6.1 Behavioral simulation

As shown in Figure 4, all supported operations execute as expected. First, the addition of 409 (0x0199) and 98 (0x0062) correctly yields 507 (0x01FB). Next, the multiplication of 2 (0x0002) by 4 (0x0004) produces 8 (0x0008). Then, a bitwise OR operation between 3 (0x0003) and 6 (0x0006) results in 7 (0x0007). Finally, performing an Arithmetic Shift Right (SRA) on −4 (0x7FFC) produces −2 (0x7FFE), verifying that the MSB is properly replicated to preserve the sign of negative numbers.

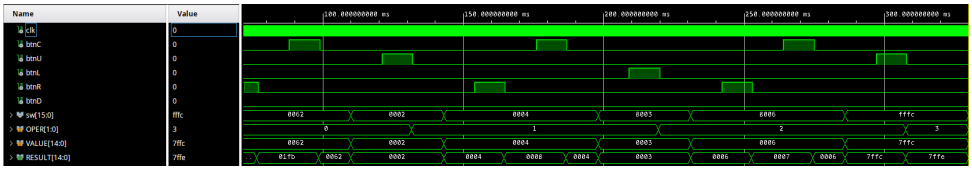


Figure 4: Simulation showing the execution of all calculator operations

Figure 5 demonstrates two consecutive addition operations, highlighting result accumulation and overflow detection. In the first operation, the result 507 (0x01FB) is stored in the register to be reused as the first

operand for the subsequent addition. In the second operation, adding 16383 (0x3FFF) the maximum 14-bit value to the stored result causes an arithmetic overflow. As expected, this condition is correctly detected and indicated by the change of the `overflow_flag` signal value.

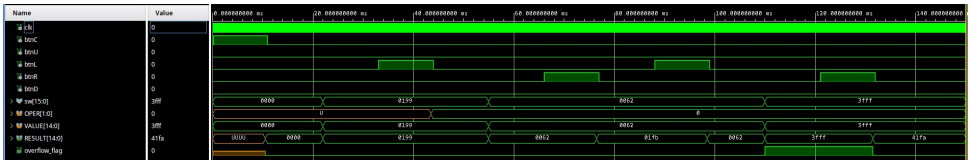


Figure 5: Simulation showing two sum operations with overflow

7 Synthesis results

Table 1 summarizes the post-synthesis FPGA resource utilization for the complete system, highlighting a minimal logic footprint of under 1% across LUTs and registers while consuming 46.23% of the available IOBs due to the physical peripherals. During synthesis, Vivado optimized the RTL by encoding the 6-state FSM in the `control_unit` with a One-Hot scheme. For the target 10 ns (100 MHz) clock period, the post-synthesis timing analysis achieves a Worst Negative Slack (WNS) of 3.082 ns, confirming a positive timing margin prior to implementation.

8 FPGA implementation

Following implementation, the setup timing analysis yields a Worst Negative Slack of 2.82 ns, confirming timing closure with a minimum achievable clock period of 7.18 ns ($f_{\max} \approx 139.28$ MHz). Table 1 indicates that logic optimization during implementation successfully reduced the Slice LUT utilization from 118 to 111.

Table 1: FPGA resource utilization

Resource	Post-Synthesis	Post-Implementation	Available	Utilization (%)
Slice LUTs	118	111	20 800	0.53%
Slice Registers (FFs)	168	168	41 600	0.40%
DSP48E1 Slices	1	1	90	1.11%
Bonded IOBs	49	49	106	46.23%
Block RAM Tiles	0	0	50	0.00%

9 Conclusions

In this project, the group designed, simulated, synthesized, and implemented a calculator in VHDL for the AMD Artix-7 FPGA on a Basys 3 board. The design uses three main units: Interface, Datapath, and Control. The calculator executes four main operations: Addition, Multiplication, Arithmetic right shift, Bitwise OR.

Behavioral simulations confirmed that all operations execute correctly and detect arithmetic overflow. The physical implementation requires few hardware resources: 111 Slice LUTs (0.53%), 168 registers (0.40%), and 1 DSP slice (1.11%). The system achieved timing closure with a Worst Negative Slack (WNS) of 2.82 ns, which allows a maximum clock frequency of 139.28 MHz.

Through this work, the group acquired the following engineering competences:

- Design hierarchical hardware architectures in VHDL.
- Develop finite state machines (FSM) and combinational logic for overflow detection.
- Write VHDL testbenches for behavioral simulation.
- Analyze Vivado synthesis, placement, and timing reports for physical FPGA deployment.